

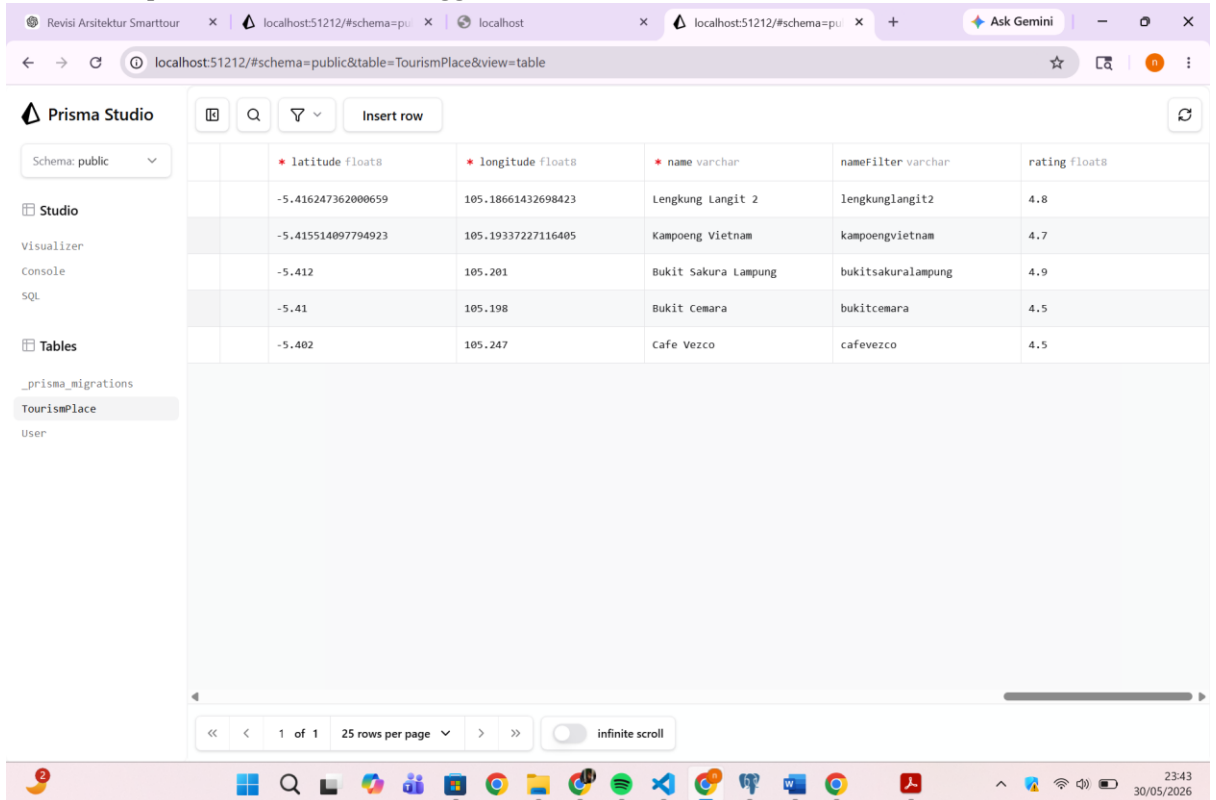
Dokumentasi tahap 2

Nama : Anisa Setia Ningsih

Kelas : IF 23 D

DOKUMENTASI TAHAP 2

1. Validasi Duplikasi Data Wisata Menggunakan nameFilter



The screenshot shows the Prisma Studio interface for a database schema named 'public'. The 'TourismPlace' table is selected, displaying 25 rows. The table has columns for latitude, longitude, name, nameFilter, and rating. The 'nameFilter' column contains normalized versions of the 'name' column, where spaces are removed and all letters are in lowercase. For example, 'Lengkung Langit 2' is normalized to 'lengkunglangit2'.

latitude	longitude	name	nameFilter	rating
-5.416247362000659	105.18661432698423	Lengkung Langit 2	lengkunglangit2	4.8
-5.415514097794923	105.19337227116405	Kampoeng Vietnam	kampoengvietnam	4.7
-5.412	105.201	Bukit Sakura Lampung	bukitsakuralampung	4.9
-5.41	105.198	Bukit Cemara	bukitcemara	4.5
-5.402	105.247	Cafe Vezco	cafevezco	4.5

Gambar 4.1 Validasi Data Wisata

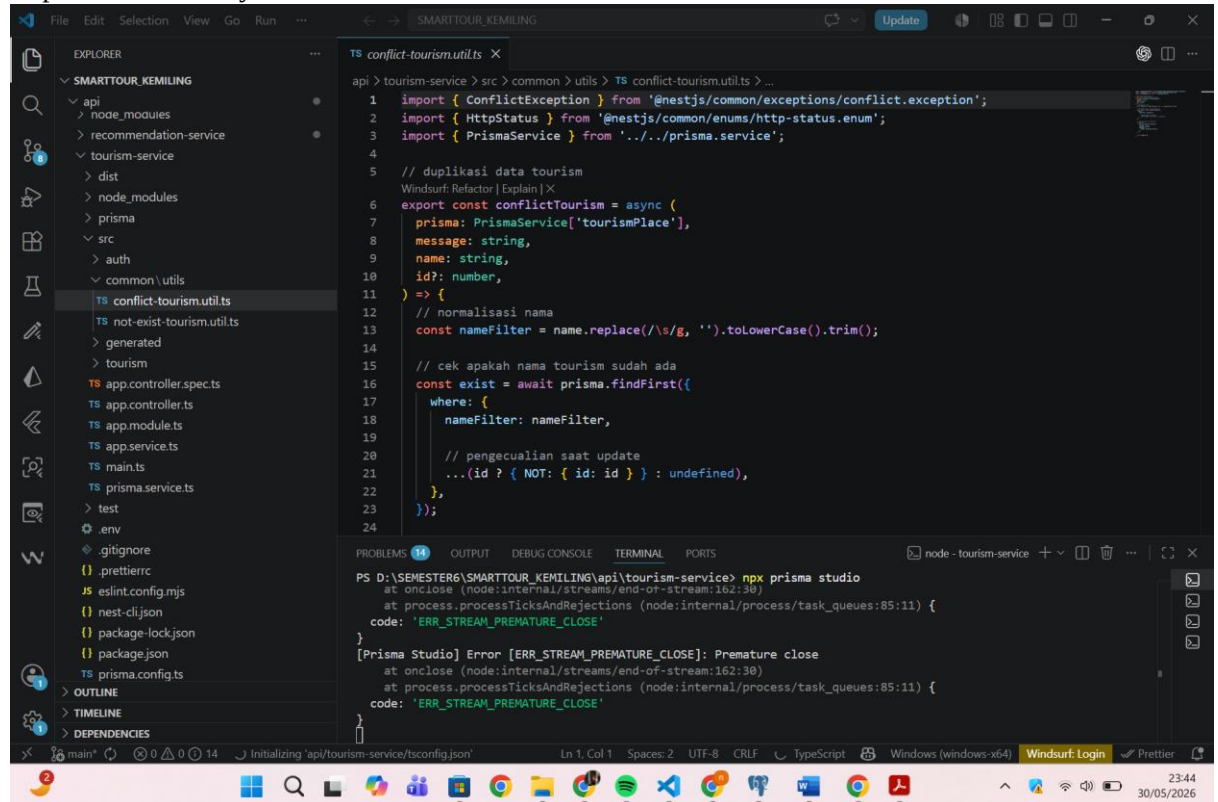
Pada tahap ini dilakukan pengembangan backend pada Tourism Service untuk mencegah terjadinya duplikasi data wisata berdasarkan nama destinasi.

Sebelumnya sistem masih memungkinkan penyimpanan data wisata yang memiliki nama sama namun ditulis dengan format berbeda, seperti perbedaan penggunaan spasi maupun huruf besar dan kecil.

- Implementasi nameFilter:
 - Sistem menggunakan field tambahan berupa nameFilter untuk melakukan normalisasi nama wisata sebelum data disimpan ke database.
 - Proses normalisasi dilakukan dengan menghapus seluruh spasi, mengubah huruf menjadi lowercase, dan melakukan trimming string.
- Pencegahan Data Duplikat:
 - Sistem melakukan pengecekan terhadap nameFilter sebelum proses penyimpanan data.

- Apabila ditemukan nilai nameFilter yang sama, data wisata akan ditolak.
- Konsistensi Data:
 - Pendekatan ini memastikan nama wisata yang sebenarnya sama tidak tersimpan lebih dari satu kali meskipun memiliki format penulisan berbeda.

2. Implementasi Utility Validasi Tourism



Gambar 4.2 Utility Validation Tourism

Untuk meningkatkan efisiensi dan keteraturan kode, validasi data dipisahkan ke dalam utility khusus.

- Utility conflictTourism:
 - Digunakan untuk melakukan pengecekan duplicate data wisata berdasarkan nameFilter.
 - Utility akan menghasilkan ConflictException apabila nama wisata sudah digunakan.
- Utility notExistTourism:
 - Digunakan untuk memeriksa keberadaan data wisata berdasarkan ID.
 - Sistem akan menampilkan error apabila data yang diminta tidak ditemukan.
- Reusable Validation:
 - Penggunaan utility membantu mengurangi pengulangan kode pada service.
 - Validasi dapat digunakan kembali pada berbagai proses backend.

3. Implementasi Error Handling Menggunakan try-catch

Pada tahap berikutnya dilakukan perbaikan struktur service menggunakan try-catch agar penanganan error menjadi lebih terstruktur.

- Penanganan `HttpException`:
 - Error validasi seperti duplicate data dan data tidak ditemukan tetap diteruskan menggunakan `throw error`.
 - Pendekatan ini menjaga pesan error tetap sesuai dengan validasi yang dilakukan.
- Penanganan Error Sistem:
 - Error lain yang tidak termasuk validasi ditangani menggunakan `BadRequestException`.
 - Sistem menampilkan response yang lebih konsisten kepada frontend.
- Konsistensi Response:
 - Seluruh error menggunakan format response yang seragam sehingga lebih mudah diproses oleh CMS maupun frontend.

4. Implementasi Environment Variable dan Konfigurasi Service

Konfigurasi backend kemudian disesuaikan menggunakan environment variable agar lebih fleksibel dan mengikuti pola pengembangan project.

- Penggunaan `.env`:
 - Sistem menggunakan `DATABASE_URL` untuk koneksi database PostgreSQL.
 - Port backend disimpan pada variabel `PORT`.
- Konfigurasi `main.ts`:
 - Backend tidak lagi menggunakan port hardcoded.
 - Service dijalankan menggunakan `process.env.PORT`.
- Kemudahan Pengelolaan:
 - Penggunaan environment variable mempermudah perubahan konfigurasi tanpa perlu mengubah source code secara langsung.

5. Pengujian Tourism Service

Pengujian dilakukan menggunakan Apidog dan Prisma Studio untuk memastikan seluruh proses backend berjalan dengan baik.

- Pengujian Create Data:
 - Data wisata baru berhasil disimpan.

• Pengujian Duplicate Data:

The screenshot shows the Apidog interface for testing a POST request to `http://localhost:3001/tourism`. The request body is a JSON object:

```
1 {
2   "name": "Cafe Vezco",
3   "description": "Cafe indoor dengan suasana nyaman yang cocok untuk bersantai dan berkumpul bersama teman maupun keluarga",
4   "isIndoor": true,
5   "latitude": -5.402,
6   "longitude": 105.247,
7   "rating": 4.5
8 }
```

The response is a 409 status code with the following JSON body:

```
1 {
2   "success": false,
3   "message": "Nama wisata sudah digunakan",
4   "metadata": {
5     "status": 409
6   }
7 }
```

The response validation shows a failure: "HTTP Status Code should be 200".

Sistem berhasil menolak nama wisata dengan nameFilter yang sama.

• Pengujian Update Data:

The screenshot shows the Apidog interface for testing a PATCH request to `http://localhost:3001/tourism/6`. The request body is a JSON object:

```
1 {
2   "name": "Bukit Cemara"
3 }
```

The response is a 200 status code with the following JSON body:

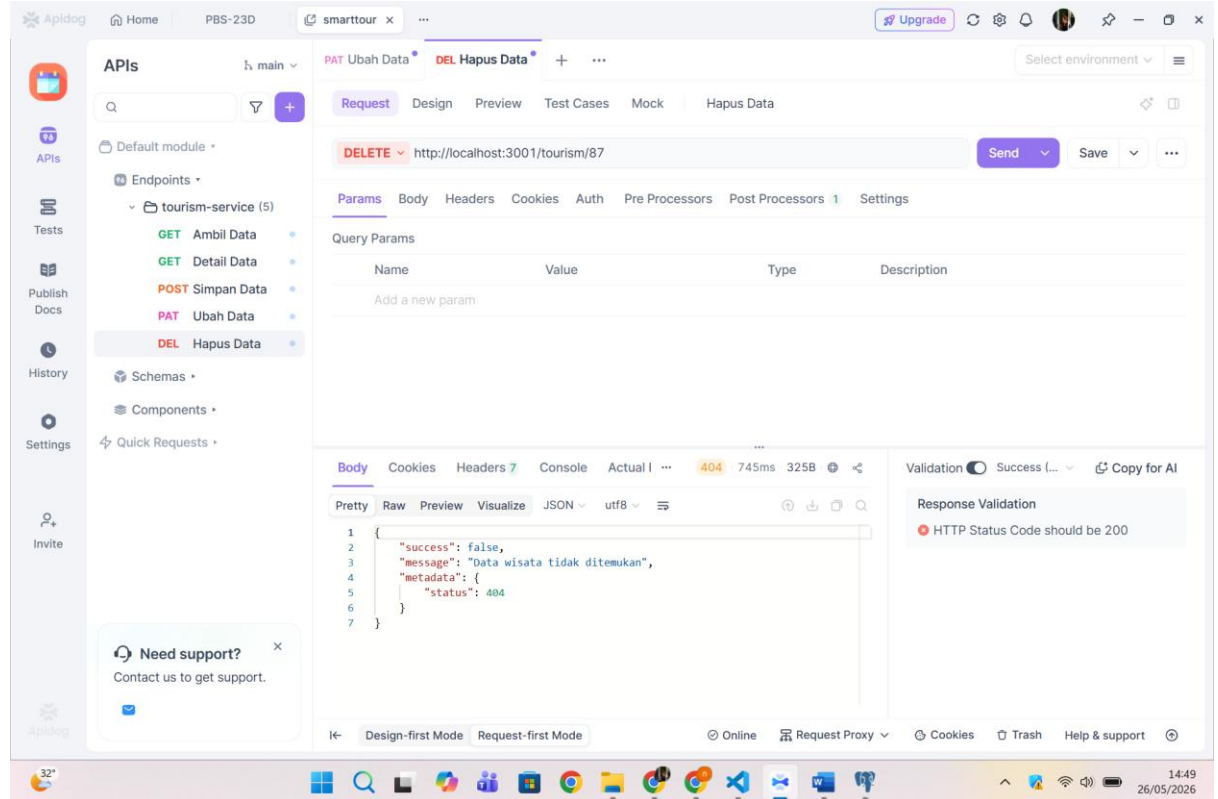
```
1 {
2   "success": true,
3   "message": "Data wisata berhasil diubah",
4   "metadata": {
5     "status": 200
6   }
7 }
```

The response validation shows a success: "Success (200)".

Update berhasil dilakukan apabila data valid.

- Pengujian Delete dan Detail Data:
Sistem berhasil menampilkan maupun menghapus data berdasarkan ID.

- pengujian Validasi Parameter ID



Sistem melakukan validasi terhadap parameter ID agar hanya menerima format angka. Apabila parameter tidak sesuai, sistem menampilkan error.

- Hasil Pengujian:

- Tourism Service berhasil berjalan dengan validasi yang lebih baik, error handling yang konsisten, serta struktur backend yang lebih rapi.